

IE3102 Microprocessors Tutorial 1 (Week 1)

1) Find the two's complement representation for the following numbers, assuming that they are represented as a 16-bit number. Write the value in both binary and hexadecimal.

- a) -93
- b) 1034
- c) 492
- d) -1094

2) Using the smallest data size possible, either a byte, a halfword (16 bits), or a word (32 bits), convert the following values into two's complement representations:

- a) -18,304
- b) -20
- c) 114
- d) -128

3) Convert the following hexadecimal values to base ten:

- a) 0xFE98
- b) 0xFEED
- c) 0xB00
- d) 0xDEAF

4) Convert the following decimal numbers into hexadecimal

- a) 256
- b) 1000
- c) 4095
- d) 42

5) Convert the following fractions into decimal numbers

- a) Binary number : 101.111
- b) Hexadecimal number : 101.111

6) Convert the following numbers into binary numbers

- a) Decimal number: 8.625
- b) Hexadecimal number : A1.E8

7) List the advantages of using the two's complement representation over the more intuitive sign-magnitude representation.

IE3102 Microprocessors Tutorial 2 (Week 2)

1) Convert the following decimal numbers into IEEE 754 single-precision format and write your answers in hexadecimal form.

- a) 0.1743129
- b) -31.5
- c) $8E-39$

2) Add the following 8-bit binary number pairs using 8-bit addition (result stored in eight bits only) and comment on whether there is an overflow if they were both unsigned numbers or they were both signed numbers.

- a) 0011 1111 and 1101 0011
- b) 0111 1111 and 0111 0011
- c) 1000 1111 and 1101 0011

3) What are the standard use of register r13, r14 and r15?

4) How many stages does the ARM7TDMI pipeline have? Name them.

5) Explain the current program state if the CPSR had the value 0x700000D3

IE3102 Microprocessors Tutorial 3 (Week 3)

Note: Before students begin on the tutorial, they have to install the Keil uVision 4 in their notebook and bring to classroom to work on the following tutorial questions. Tutors are supposed to run through the uVision setup and highlight the key functions, windows and use Q1 below to run through the uVision.

1) The factorial program is given as follows:

```
AREA Prog2, CODE, READONLY
ENTRY
MOV r6, #10           ; load 10 into r6
MOV r4, r6             ; copy n into a temp register
loop SUBS r4, r4, #1    ; decrement next multiplier
    MULNE r6, r4, r6    ; perform multiply
    BNE loop           ; go again if not complete
stop B stop
END
```

- a) **Using the Disassembly window (Keil μ Vision4)**, write out the first six machine codes (32-bit instructions) for the above program in hex format.
 - b) Change the value in register r6 at the start of the program to 12. What value is in the register r6 when the code terminates? Verify that this hex number is correct.
- 2) Show three different ways to clear all the bits in register r12 to zero. You may not use any registers other than r12. **Using the Disassembly window (Keil μ Vision4) to show the answer.**
- 3) Use an assembler directive to assign register r6 to the name bouncer.
- 4) What instruction puts the ASCII representation of the character “R” in register r11?
- 5) Give the directive to reserve a block of zeroed memory, holding 40 words and labeled coeffs.
-

IE3102 Microprocessors Tutorial 4 (Week 4)

- 1) Describe the contents of register r13 after the following instructions complete, assuming that memory contains the values shown below. Register r0 contains 0x4000 1000, and the memory system is little-endian. **Use Keil to run through the following instructions. Set the target to LPC 2104 from now onward, and store the content into the below address first before running the LDR code.**

Address	Contents
0x4000 1000	0x06
0x4000 1001	0xFC
0x4000 1002	0x03
0x4000 1003	0xFF

- a) LDRSB r13, [r0]
 - b) LDRSH r13, [r0]
 - c) LDR r13, [r0]
 - d) LDRH r13, [r0]
- 2) Calculate the effective address of the following instructions if register r3 = 0x4000 1000 and register r4 = 0x20:
- ☐ STRH r9, [r3, r4]
 - ☐ LDRB r8, [r3, r4, LSL #3]
 - ☐ LDR r7, [r3], r4
 - ☐ STRB r6, [r3], r4, ASR #2
- 3) Write a program that sums word-length values in memory, storing the result in register r2. Include the following table of values to sum in your code:

TABLE DCD 0xFEBBA, 0x1234, 0x8888

DCD 0x13, 0x8080808, 0xFFFF

(Use Keil to program and display your result in the register window)

IE3102 Microprocessors Tutorial 5 (Week 5)

1) What constant would be loaded into register r7 by the following instructions?

- a) MOV r7, #0x8C, 4
- b) MOV r7, #0x42, 30
- c) MVN r7, #2
- d) MVN r7, #0x8C, 4

(Use Keil to demonstrate the MOV instructions of the above)

2) Whenever possible, use the byte rotation scheme described in the lecture notes for the MOV instruction to generate an instruction needed to load the following constants into register r2. If it is not possible to use a MOV instruction, provide an alternative instruction that do the job.

- a) 0xA400
- b) 0x7D8
- c) 0x17400
- d) 0x1980

(Use Keil to demonstrate the use of MOV or other instructions to load the above constant to r2)

3) Without using the MUL instruction, give instructions that multiply register r4 by :

- a) 135
- b) 255
- c) 18
- d) 16,384

and place your result in register r0. Your solution for each part should only have two or less instructions.

(Use Keil to demonstrate how the above can be carried out)

4) Find the maximum value in a list of 32-bit values located in memory. Assume the values are in two's complement representations. Your program should have 50 values in the list.

(Use Keil to validate your answer. Students can choose their own registers for their program)

5) Convert the following C iteration statements into assembly language. For example, in C language, the following *for* loop:

```
for (i = 10; i > 0; i--) {  
    .... // loop body  
}
```

can be written in ARM assembly language as

```
    MOV r0, #10    ; r0 is used to store i  
Loop    . . . ;loop body  
    SUBS r0, r0, #1 ;loop 10 times  
    BNE loop
```

Write the assembly language equivalent for the following using r0 for i, r1 for j and r3 for k:

- a) `for (i=0; i<10; i++) { ...}`
- b) `for (i = 1; i < 11; i++) { //nested loop
 for (j = 5; j > 0; j--) {
 } }`
- c) `for (i = 18; i >= 3; i--) {
 for (j= 16; j < 40; j++) {
 for (k = 0; k < 30; k++) {
 }}}`

Hands-on CA#1 question will be released in the NTULearn on Week #6 (45 mins). The hands-on CA must be done individually without discussion. Student can still refer to their lecture/tutorial notes. The complete .s (asm file) and the results windows (registers/memory) will be deposit in the tutorial folder (NTULearn). It will be marked by TA and returned to students for reference.

IE3102 Microprocessors Tutorial 6 (Week 7)

1. Write an assembly program to construct a table of Fibonacci numbers for $n = 0$ to 20. The starting address of the table in memory is 0x40000000. In mathematics, the **Fibonacci numbers** or **Fibonacci series** or **Fibonacci sequence** are the numbers in the following integer sequence. Assume that the values are 32-bit data.

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...

By definition, the first two numbers in the Fibonacci sequence are 0 and 1, and each subsequent number is the sum of the previous two. Hence, the sequence F_n of Fibonacci numbers is defined by the recurrence relation:

$$F_n = F_{n-1} + F_{n-2}$$

with seed values, $F_0 = 0$ and $F_1 = 1$.

(Write the assembly program in Keil and show where the results are stored in memory)

2. Write an assembly program to examine a table (a list of 32-bit values) sequentially for a match with a search key. Store the search key as a new entry if no match is found by adding it to the end of the list (highest address).

Assume that the first value in the list indicates the length of the list and load it to register r2 (assume =4). Register r0 initially point to the starting address of the list. It will eventually point to the address of the matched item if there is a match. Register r1 contains the search item (assume =0x9ABCDEF0). You need to include the initialization file: tut7q2_L2104.ini .

(Write the assembly program in Keil and use different search item to test out your code)

IE3102 Microprocessors Tutorial 7 (Week 8)

1. State the validity for each of the following instructions. Explain why for those that is not valid.

- a) STM r4, {r4-r6}
- b) LDM r4, {r4, r7}
- c) STMIA r5!, {r5, r4, r9}
- d) LDMDA r2, {}
- e) STMDB r15!, {r0-r3, r4, lr}

(Use Keil to confirm your answer and find a way to rectify the code if it is not valid)

2. If register r6 holds the address 0x8000 and you execute the instruction

STMIA r6, {r7, r4, r0, lr}

Write the memory address now holds the value that was in registers r0, r4, r7 and lr respectively.

(Use Keil to illustrate your answer and understand the data movement)

3. Assume that the memory and registers r0-r3 of an ARM processor appears as follows:

Address

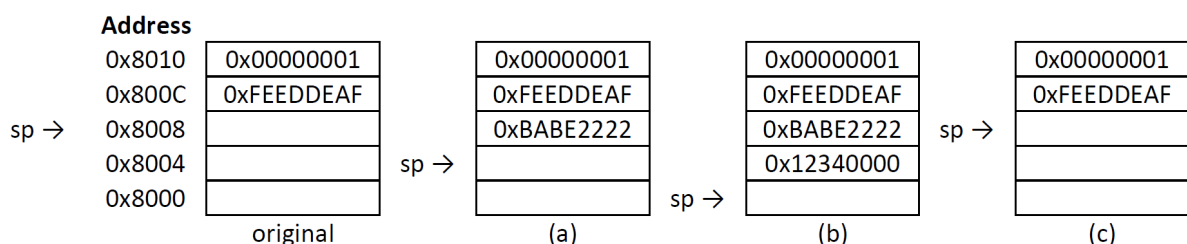
0x8010	0x00000001
0x800C	0xFEEDDEAF
0x8008	0x00008888
0x8004	0x12340000
0x8000	0xBABE0000

Give the contents of the memory and registers that have been changed after executing the instruction

LDMIA r3!, {r0, r1, r2}

(Use Keil to illustrate your answer and understand the data movement)

4. Suppose that a stack appears as shown in the original diagram below. Give the instruction(s) that would push or pop data so that memory appears in the order shown. In other words, what instruction would be necessary to go from original state to that shown in (a) and then (b) and then (c)?



IE3102 Microprocessors Tutorial 8 (Week 9)

1. Why is it that we may get into an infinite loop when one subroutine calls another subroutine?
 2. Write an ARM subroutine in assembly to compute factorial of n, using a full descending stack, for each of the following methods of passing parameters. Assume that register r0 contains n (=10) and the result will be stored in register r1.
 - a) Passing parameters to/from the subroutine using pass-by-register.
 - b) Passing parameters to/from the subroutine using pass-by-stack.**(Use Keil to illustrate the above program)**
 3. Write an ARM assembly program to sum the square of 4 numbers ($\text{sum} = x1*x1 + x2*x2 + x3*x3 + x4*x4$). The main program should call a subroutine to add a list of number. This subroutine would then call another subroutine to perform the square operation. Test it out using Keil uVision and observe the content of registers sp, lr and pc during subroutine calls and returns. Let register r0 be the pointer to the numbers and register r1 contains the final result.
(Use Keil to perform this task)
-

IE3102 Microprocessors Tutorial 10 (Week 10)

1. Based on the following exception handlers addresses, complete the partial Vector Table with the appropriate instructions at their respective exception address.

Exception	Handler Address
...	...
Software Interrupt	0x40000000
Undefined Instruction	0x3020
Reset	0x40001000 Assume that this 32-bit address is stored at 0xEFC

address	Exception	Instruction
0x10	...	...
0x0C	...	...
0x08	Software interrupt	?
0x04	Undefined instruction	?
0x00	Reset	?

2. Name three ways in which FIQ interrupts are handled more quickly than IRQ interrupts.

3. How many external interrupt lines does the ARM7TDMI have? If you have eight interrupting devices, how would you handle this?

IE3102 Microprocessors Tutorial 10 (Week 11)

1. Which bit in the registers CPSR and SPSR indicate whether you are in ARM or THUMB state? [optional]
2. Explain how the branch instruction *BX rd* is used to switch between the ARM and THUMB states. [optional]
3. Write the THUMB instructions that are equivalent to the following ARM instructions.
 - a. ADDS r0, r3, r2, LSL #2
 - b. LDR r1, [r4, r5, LSL #2]

(Homework: Use Keil to show how to break these ARM code to THUMB code)

4. The following program fragment is written in ARM code. Determine what it is doing and compute the code size of this fragment. Convert it into a THUMB program fragment and also determine its code size.

```
    ;IN: r0 (value) , r1 (divisor)
    ;OUT: r2 (remainder), r3 (quotient)
    CODE32
    MOV     r0, #10
    MOV     r1, #3
    MOV     r3, #0
loop SUBS   r0,r0, r1
    ADDGE   r3, r3, #1
    BGE     loop
    ADD     r2, r0, r1
```

(Use Keil to perform the above)

5. Write an assembly program that will call a subroutine (written in THUMB code) with register r0 = 5 and register r1 = 3. The THUMB subroutine computes the sum of the square of the values stored in register r0 and register r1. It should place the result in register r2. You should write code (veneer) that will enable the switch to different states and also ensure that any other registers used in the subroutine are preserved upon entry and restored before return.

(Homework: Use Keil to perform the above)

6. Write a short C program that declares 3 integers called x, y and z and determine the maximum value among them. Print out this maximum value to UART0. To test the program, let x = 9, y = 4 and z = 12.

- a) Write the maximum program as an inline assembly function.
- b) Write the maximum program as an embedded assembly function.

7. Below is a main program written in C. It calls a subroutine to append two strings into one. Write the subroutine in a separate ARM assembly file. Test the program using the Keil µVision 4 simulator.

```
#include <stdio.h>
#include <LPC21xx.H> /* LPC21xx definitions */
extern void Init_serial(void);
```

```

extern void appendStr(const char *s1, const char *s2, char *s3);
/* main program */
int main (void) {
    const char *s1 = "Hello";
    const char *s2 = "World";
    char s3[30];          /*s3 should be "HelloWorld" eventually */
    Init_serial();        /* initialize the serial interface */
    appendStr(s1,s2,s3);  /* call the asm subroutine */
    printf ("Appending %s with %s produces %s\n", s1, s2, s3);
    while (1) { /* An embedded program does not stop and */
        ; /* ... */ /* never returns. We use an endless loop. */
    } /* Replace the dots (...) with your own code. */
}

```

Hands-on CA #2 question will be released in the NTULearn on Week #12 (45 mins). The hands-on CA must be done individually without discussion. Student can still refer to their lecture/tutorial notes. The complete .s (asm file) and the results windows (registers/memory) will be deposit in the tutorial folder (NTULearn). It will be marked by TA and returned to students for reference.